VisionMate

A voice-first assistant for blind and low-vision users in seven Indian languages

Final Report

1. Solution

Problem. Most digital services assume the user can see the screen, and many assume the user reads English. Blind and low-vision people in India therefore struggle with everyday tasks such as reading a printed label or menu, understanding a message in another language, finding a place, noting something down, or calling for help quickly. Existing assistants are rarely designed for Indian languages, and native accessibility apps need installation and device-specific permissions.

What we built. VisionMate is a mobile-first, installable web application (PWA) that a user operates mainly by voice. The user presses and holds a Talk button, or types, and VisionMate answers aloud in the chosen language: English, Hindi, Tamil, Telugu, Kannada, Malayalam or Punjabi. One interface combines a voice assistant, a camera reader (Vision Lens), a translator, an approximate walking-route estimator (Go), notes and reminders, trusted contacts, and an always-visible SOS button.

Why a web app. It runs on any modern mobile browser, needs no store download, can be installed to the home screen, and opens its app shell offline through a service worker. The trade-off is that a website cannot draw over other apps or read hardware volume buttons. VisionMate says so openly instead of imitating those features.

Key outcomes. A single guest-ready app that (a) works immediately without sign-in, (b) uses Indian-language services (Bhashini and Anuvaadhub) together with Gemini, (c) keeps all private API keys on the server, and (d) asks consent before using the camera, microphone or location.

2. Approach

2.1 Design principles. Four rules guided every screen and service call:

- **Voice first.** Press-and-hold Talk, browser speech recognition, optional recorded speech sent to Anuvaadhub, a typed fallback, and queued spoken replies that never overlap.
- **High contrast and large controls.** Dark theme with a single orange action colour, plain-language prompts, and a fixed bottom bar (Back, Home, Go, Lens, Voice, Me).
- **Consent before sensing.** Camera, microphone and location start only when the user starts that feature; route lookup needs a ticked consent box.
- **Honest limits.** Screens warn that AI answers can be wrong and that the app is not a mobility or safety aid.

2.2 System architecture. The frontend is plain HTML, CSS and JavaScript ES modules (about 4,000 lines including server code) with no bundler. Every call to an AI provider goes through one of three JWT-protected Supabase Edge Functions written in TypeScript for Deno. The browser never holds a provider secret.

Layer	Technology	Role
Client	HTML, CSS, JS modules; Web Speech; MediaDevices; Leaflet; service worker; web manifest	Screens, voice, camera, map, offline app shell, installable PWA
Identity	Firebase Authentication (Google, email); Supabase anonymous session for guests	Optional accounts; guests still reach protected AI functions
Server	Edge Functions: gemini-assist, bhashini-translate, anuvaadhub-inference	Hold secrets; validate sizes, formats and methods
Data	Supabase Postgres with row-level security (8 tables); optional Firebase Realtime Database mirror	Profiles, notes, contacts, reminders, settings, chat, activity
Language AI	Bhashini, Anuvaadhub, Gemini	Translation, speech, OCR, assistant and vision answers
Routing	Photon geocoding, Valhalla pedestrian routing, OpenStreetMap tiles	Approximate distance, time and route line

2.3 Use of Bhashini and Anuvaadhub. The Translator screen lets the user choose a provider. For **Bhashini**, the bhashini-translate function reads the user ID, API key and pipeline ID from server secrets, asks the ULCA model-pipeline service for a translation pipeline for the requested language pair, then calls the returned HTTPS callback with the text (up to 4,000 characters) and returns the translated string. Failures at each step return distinct errors. For

Anuvaadhub, an approved catalogue of 63 model entries is loaded by the anuvaadhub-inference function: 42 machine-translation, 7 speech-recognition, 7 OCR and 7 text-to-speech models covering the seven languages, each with a rate limit of 50 requests per minute. The browser limits input to 1 to 50 words for translation, 30 words for speech, and 5 MB for recorded WAV audio (converted to 16 kHz in the browser).

2.4 Use of Gemini. The gemini-assist function handles three jobs: answering a text prompt, answering a question about one image frame (limited to about 4 MB and JPEG, PNG or WebP), and interpreting a spoken command into an app action. In speech mode it first translates the reply into the selected language and then generates audio with a Gemini text-to-speech model. The device voice is used as a fallback when a matching installed voice exists.

2.5 Data and privacy. Guests keep notes, reminders, settings and chat history only in the browser. Signed-in users sync to Supabase Postgres, where row-level security restricts each row to its owner's Firebase user ID; a small Firebase Cloud Function adds the role claim Supabase needs. A people-privacy setting asks the model not to describe or identify people. Phone calls and external links always need the user's confirmation.

3. Features

Feature	What the user can do	How it works
Voice agent	Hold Talk and speak, or type. Say commands such as "open Vision Lens" or "write a note". Hear the reply spoken.	Browser speech recognition or Anuvaadhub ASR on recorded WAV; Gemini maps the request to an app action; serialized speech queue.
Vision Lens	Start the camera, switch front or rear, zoom, go fullscreen, then ask Gemini to read a menu, label or document from one still frame.	MediaDevices API with hardware or digital zoom; the frame is sent only when the user asks; optional periodic snapshots only while enabled.
Translator	Type or dictate a short text and translate it into any of the seven languages; choose Bhashini or Anuvaadhub.	Edge Functions call the provider; provider choice and target language are saved in settings.
Speech output	Have a phrase spoken in a chosen language with a female or male voice.	Anuvaadhub TTS (up to 30 words) or Gemini generated speech; device voice fallback.
Go (routes)	Enter a destination, give consent, and see approximate walking distance, time and route line on a map.	Photon geocoding and Valhalla routing via routes.js; polyline decoding; Leaflet with OpenStreetMap tiles.
Notes and reminders	Save a spoken or typed note and set reminders; open supported apps or call a contact after confirming.	Local storage first; Supabase sync when signed in; browser notifications when allowed.
Emergency	SOS button on every screen opens emergency support: show location, call a trusted contact.	Phone hand-off with confirmation; emergency entry cached by the service worker.
Profile and settings	Edit profile, language, voice, theme and privacy options; review recent chat history; check permissions.	Guest profile in the browser; account data under row-level security.
Guest mode and PWA	Use everything immediately without an account; install to the home screen.	Anonymous Supabase session for protected functions; manifest and service worker.

4. Results

VisionMate was implemented end to end as a working web app. Behaviour was checked manually on mobile-sized screens. The repository has no automated test suite, so the outcomes below are described as observed behaviour rather than benchmark scores.

Area	Observed result
Guest start	The app opens and works without sign-in; notes, settings and chat history are saved locally in the browser.
Permissions	Camera, microphone, location and notification access are requested only after the user starts the related feature. A permissions screen shows the state of each.
Vision Lens	The camera preview starts on request, shows a clear on or off status, and supports zoom and fullscreen. Analysis sends a single still frame on demand.
Go	A route estimate is blocked until the consent box is ticked. The result is labelled informational and not turn-by-turn.
Languages	Seven languages work for greetings, spoken replies and translation. Provider limits (50 words for translation, 30 for speech, 5 MB for recordings) are enforced with clear messages.
Error handling	When cloud sign-in was switched off, the chat showed a plain explanation instead of failing silently, and local features kept working.
Security	Provider keys exist only as server-side secrets; JWT verification is enabled on all functions; Postgres tables use row-level security.

Overall result. The project delivers the main goals: hands-free control, reading from the camera, translation across seven Indian languages through Bhashini and Anuvaadhub, approximate route estimates, notes, and one-tap emergency access, all in a single installable app that also works for guests.

Observed constraints. Image and AI answers can be wrong; routes are approximate and not checked for sidewalks or closures; browser speech recognition and installed voices differ between devices; reminders may not ring after the browser closes; overlay and hardware-button shortcuts need a separate native Android app; and Anuvaadhub model access is time-limited and must be renewed. VisionMate should be used alongside, never instead of, a cane, guide dog or trusted assistance.